Diseño de interfaces web

Diseño de interfaces web
- Aplicaciones web
  - Elementos
  - Middleware
- Herramientas de desarrollo web
  - HTML5
  - CSS3
  - ECMAScript
  - Páginas de interés
- DOM (*Document Object Model*)
- Interfaces web en dispositivos móviles
  - Diferencias notables entre dispositivos móviles y de sobremesa
  - Herramientas de desarrollo y pruebas
  - Opciones de desarrollo
- *Website* dedicado
  - HTML para móviles
  - CSS para móviles
  - JavaScript para móviles
    - Geolocalización
  - Optimizaciones para móviles
    - Optimización de formularios
    - *CSS Sprites*
  - Mejores prácticas
- *Responsive Web Design*
  - Contenidos flexibles
    - Medidas flexibles
    - Medidas *Responsive*
  - *CSS Media Queries*
  - Imágenes flexibles
    - Antes de HTML 5.1
    - Imágenes flexibles en HTML 5.1
  - Workflow
  - Hojas de estilos

Asynchronous Javascript And XML (AJAX)
- Cross-Origin HTTP Request (CORS)
- Peticiones AJAX
  - *Legacy*
    - IE5, IE6
    - Chrome, IE7+, Firefox, Safari, Opera
  - ECMAScript 6 (2015)

WAI-ARIA (*Web Accessibility Initiative - Accessible Rich Internet Applications*)
- Elementos
  - Roles
    - *Landmark roles*
    - *Document structure roles*
    - *Widget roles*
    - *Live region roles*
  - Estados y propiedades
    - Atributos de *widgets*
    - Atributos de relación

Atributos de regiones activas
Reglas de uso

Aplicaciones web

Son aplicaciones Cliente/Servidor. Originalmente la web no se diseñó para desarrollar aplicaciones sino para compartir recursos (documentos e imágenes).

Elementos

Cliente	Middleware	Servidor
Navegador	Http	Genérico

Middleware

Lógica de intercambio de información entre aplicaciones. Asiste a una aplicación para interactuar o comunicarse con otras aplicaciones, paquetes de programas, redes, hardware o sistemas operativos.

Simplifica el trabajo de los programadores en la tarea de generar las conexiones y sincronizaciones que son necesarias en los sistemas distribuidos. De esta forma, se provee una solución que mejora la calidad de servicio, así como la seguridad, el envío de mensajes, la actualización del directorio de servicio...

Funciona como una capa de abstracción de software distribuida, que se sitúa entre las capas de aplicaciones y las capas inferiores (sistema operativo y red) proporcionando una API para la fácil programación y manejo de aplicaciones distribuidas.

Herramientas de desarrollo web

HTML5

Versión más reciente de HTML estandarizada por el W3C. Se encarga de la distribución de los elementos en el documento.

CSS3

Versión más reciente de CSS estandarizada por el W3C. Se encarga de la apariencia de los elementoss del documento.

ECMAScript

Variedad de JavaScript estandarizada por ECMA. Añade interacción.

Páginas de interés

- https://www.w3.org/
- https://validator.w3.org/
- https://caniuse.com/
- https://www.w3schools.com/
- https://developer.mozilla.org/es/
- https://www.quirksmode.org/

- https://alistapart.com/
- https://www.sitepoint.com/

DOM (*Document Object Model*)

Es esencialmente una interfaz de plataforma que proporciona un conjunto estándar de objetos para representar documentos HTML, XHTML y XML, un modelo estándar sobre cómo pueden combinarse dichos objetos, y una interfaz estándar para acceder a ellos y manipularlos.

A través del DOM, los programas pueden acceder y modificar el contenido, estructura y estilo de los documentos HTML y XML, que es para lo que se diseñó principalmente.

Permite el acceso dinámico a través de la programación para acceder, añadir y cambiar dinámicamente contenido estructurado en documentos con lenguajes como ECMAScript (JavaScript). El responsable del DOM es el World Wide Web Consortium (W3C).

Interfaces web en dispositivos móviles

Hoy en día casi todo el mundo posee un dispositivo móvil pero no un pc de sobremesa o portátil. El número de accesos a internet mediante dispositivo móviles a aumentado llegando incluso a superar los accesos de equipos de sobremesa, por lo que es importante orientar el diseño de las interfaces a los primeros para evitar tener que rediseñarlas para adaptarlas.

Diferencias notables entre dispositivos móviles y de sobremesa

Pantalla	Conexión	Interacción	Hardware	Sensores
Tamaño	Velocidad	Gestos	CPU	Cámara
Orientación	Consumo de datos	Teclado	Memoria	GPS
			Batería	

Herramientas de desarrollo y pruebas

- Simuladores y emuladores: https://www.browserstack.com/
- Remote Labs:
  - https://firebase.google.com/docs/test-lab/
  - http://developer.samsung.com/rtlLanding.do
- Extensiones para navegador en escritorio.
- Prototipado (skins o sw específico)
- IDEs
- W3C mobileOK checker

Opciones de desarrollo

- ***Website* dedicado**:
  - Sitio optimizado para móvil.
  - 2 URLs con el mismo contenido.
  - Configuración:

        - Cliente:
            - Cookies
        - Servidor:
            - Mime-types
            - Redirección
        - Detección móvil:
            - User-agent: Cadena de caracteres que identifica el navegador y su versión.
- ***Responsive Web Design***:
    - Se adapta el *site* al tamaño de pantalla.
    - 1 URL

Website dedicado

HTML para móviles

Antes los dispositivos estaban muy limitados por las tecnologías de sus fabricantes, provocando que los navegadores de modelos diferentes utilizaran distintas versiones de lenguajes de marcado web. A día de hoy la mayoría de dispositivos móviles (comúnmente *smartphones*) son capaces de utilizar HTML5.

Engines:

- Webkit (Safari iOS, Android...).
- Gecko (Firefox mobile).
- Blink (Google Chrome (v28+), Edge, Brave, Opera v15+).
- Presto (Opera mobile).
- Trident (IExplorer mobile).
- EdgeHTML (Edge).

CSS para móviles

```
<link rel="stylesheet" type="text/css" href="file.css"
media="mediatype and|not|only (media feature)" />
```

Media queries (CSS3):

- Media type: screen
- Media feature:
    - `max-device-width` / `min-device-width`.
    - `max-width` / `min-width`.
    - `orientation`.
    - `aspect-ratio`.
    - `min-device-pixel-ratio` / `max-device-pixel ratio` (no estándar).
    - `resolution` / `min-resolution` / `max-resolution` (estándar).

JavaScript para móviles

Es un leguaje que va a ser interpretado, se va a interactuar co él y va a generar eventos. Por tanto, consume:

- Batería.
- Ancho de banda.
- Cache.

Supone un coste computacional en dispositivos no muy modernos:

- Recuperando diálogos:
  - `alert`
  - `confirm`
  - `prompt`

Para reducirlo pueden utilizarse librerías *mobile*, aunque perjudicarían el tamaño, los tiempos de carga y la inicialización.

Puede utilizarse para realizar *Browser Detection* y utilizar funciones de los navegadores.

Geolocalización

Obtención de la localización del usuario:

- Cliente:
  - GPS A-GPS.
  - Información de celda.
  - Posición WiFi.
- Servidor:
  - IP.
  - Operador.
  - Idioma.
  - WiFi AP.
- Preguntar al usuario.

En JavaScript: Objeto `navigator.geolocation` (estándar W3C). El usuario configura los permisos en el navegador. Asíncrono.

Optimizaciones para móviles

Optimización de formularios

- Simplificar formularios incluyendo sólo detalles importantes.
- Usar una disposición vertical de los elementos.
- Usar menús o listas desplegables frente a radioButtons o checkBoxes.
- Utilizar el atributo `type` más adecuado en elementos `input`.

CSS Sprites

Abrir una conexión con el servidor para cargar imágenes (`<img src="">`) es un proceso costoso. Si existen una gran cantidad de imágenes pequeñas o iconos es recomendable crear una única imagen de mayor tamaño combinando las imágenes pequeñas y restringir la visualización a zonas concretas.

Mejores prácticas

- Desarrollar código simple.
- Reducir el número de peticiones HTTP.
- Minimizar tamaño:
  - Eliminar espacios.
  - No cargar recursos no usados.
  - Máxima compresión de imágenes.
- Evitar usar sombras o gradientes.
- Usar peticiones AJAX si es posible (lazy loading).
- Evitar `try/catch`, `with`, `eval`, variables globales, *pop ups*...
- Concentrar cambios en DOM en una función.

Responsive Web Design

Se diseña una página inicialmente para móvil y a media que aumenta el tamaño de la pantalla de los dispositivos se aumenta el tamaño de los elementos o se modifica su colocación.

El **diseño web adaptativo** sigue tres principios:

- *Fluid Layout*: Todos los elementos deben ser fluidos o flexibles y adaptarse al tamaño de la pantalla.
- *Media Queries*: Para adaptar y ejercer pequeños cambios al diseño de los elementos dependiendo del dispositivo.
- Imágenes flexibles: La imágenes también se adaptan al layout flexible.

Contenidos flexibles

Eliminar unidades absolutas y utilizar unidades relativas usando la fórmula `target / context = result` siendo:

- *Target*: Tamaño deseado para el elemento.
- *Context*: Tamaño del elemento contenedor.
- *Result*: Tamaño relativo a usar.

Medidas flexibles

Se establece el tamaño de fuente al tamaño por defecto del navegador (normalmente 16px).

Se utiliza unidades relativas para escalar los tamaños fuente (em/rem):

- em: Escala respecto al contenedor padre.
- rem: Escala respecto al contenedor raíz.

```
body { font-size: 100% ; }
h1 { font-size: 1.5em; /* 24 px / 16 px */ }
p { font-size: 0.875em; /* 14 px / 16 px */ }
```

Medidas *Responsive*

Se utilizan *viewport units*: Unidades relativas a la parte visible del navegador. 1 *viewport unit* corresponde a un 1%.

- vh: *viewport height*.
- vw: *viempor width*.
- vmin: Menor tamaño (alto o ancho) del *view port*.
- vmax: Mayor tamaño (alto o ancho) del *view port*.

El navegador calcula el tamaño de fuente de forma dinámica:

```
html { font-size: calc(1em + 1vw); }
/* Si vw = 0, font-size = 1em */
/* Cuanto más grande es la parte vw, el texto se hará más grande en pantallas grandes */
```

CSS Media Queries

Se utiliza la regla `@media` para incluir un bloque de propiedades CSS solo si una determinada condición es verdadera.

```
@media condition {
  /* CSS Blocks */
}
```

Imágenes flexibles

Adaptar las imágenes (u otros elementos multimedia) al tamaño de los dispositivos. HTML 5.1 estandariza esta solución. Para lograrlo algunos navegadores soportan el uso de imágenes vectoriales (SVG...).

Antes de HTML 5.1

```
img { /* Style */ }
```

- Funciona en la mayoría de los navegadores modernos.
  - No funciona en versiones antiguas de IE!
- Escala imágenes al tamaño del elemento contenedor.
- Aplicable a otros elementos (*video*, *object*, etc).

Imágenes flexibles en HTML 5.1

- Selección en base a densidad de píxeles:
  - Retina / 4K / Ultra HD.
  - x device pixel ratio
  - No soportado por IE, Opera Mini, Android Browser v4.4.

```
<img src="im_300w.jpg" srcset ="im_300w.jpg 1x, im_600w.jpg 2x" width="300" height="300"
alt="Description"/>
```

- Selección en base a *viewport*:

    - w tamaño de imagen.
    - vw tamaño relativo de *viewport*.

```
<img src ="im_300w.jpg" srcset ="im_300w.jpg 300w, im_600w.jpg 600w" sizes="(max-width: 640px) 100vw, (max-width: 960px) 75vw, 300 px" alt="Description"/>
```

- Selección por *breakpoint*:

```
<picture>
  <source media="(min-width=960px)" srcset ="im-large.jpg">
  <source media="(min-width=575px)" srcset ="im-medium.jpg">
  <img src="im-small.jpg" alt="Desc"/>
</picture>
```

Workflow

- Opción 1: adaptar el diseño para *Desktop*:
    - No recomendada.
- Opción 2: *Mobile First*:
    - Comenzar con el diseño para móvil.
    - Optimizar para pantallas más grandes.

> *The absence of a media query is in fact, the first media query.* ~Bryan Rieger.

Hojas de estilos

- Incluir varias hojas de estilos en el documento HTML:

```
<link href="default.css" type="text/css" rel="stylesheet" media="screen "/>
<link href="desktop.css" type="text/css" rel="stylesheet" media="only screen and (min-device-width: 1024px) and (max-width: 989px)"/>
```

- Incluir una hoja de estilo que incluye *Media Queries*:

```
@media screen and (min-width: 1024 px) {
/* Ajustes menores */
section ul li { width: 20%; }
}
```

Asynchronous Javascript And XML (AJAX)

AJAX es la combinación de objetos/funciones del navegador para:

- Realizar peticiones de datos con Javascript.
- Mostrar dichos datos con HTML DOM.

Permite actualizar páginas web de forma asíncrona sin recargar la página completa.

Cross-Origin HTTP Request (CORS)

Un recurso de un dominio A hace peticiones de otro recurso a un dominio B. Habitual para:

- Ficheros de estilos CSS.
- Imágenes.
- Scripts Javascript.

Por motivos de seguridad, los navegadores no permiten realizar peticiones HTTP a otros dominios desde scripts.

Peticiones AJAX

Legacy

IE5, IE6

```
xhttp = new ActiveXObject("Microsoft.XMLHTTP");
```

Chrome, IE7+, Firefox, Safari, Opera

```
xhttp = new XMLHttpRequest();
```

- Crear una petición:

```
xhttp.open(method, url, async);
// method: GET, POST, PUT, DELETE...
// async: true/false
```

- Enviar una petición:

```
xhttp.send(); // GET
xhttp.send(string); // POST
```

- Procesar respuesta:

```
xhttp.onreadystatechange = callback;
/* callback: función que se ejecutará cada vez que cambie el estado de la propiedad
readyState */
```

- Estados posibles de `readyState`:
  - 0: petición no inicializada.
  - 1: conexión establecida con el servidor.
  - 2: petición recibida.
  - 3: petición procesándose.
  - 4: petición terminada y respuesta preparada.
- Otras propiedades del objeto `XMLHttpRequest`:
  - `status`: código de estado devuelto por el servidor tras procesar la petición (200, 400, 404...).
  - `statusText`: texto asociado al código de estado.

```
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    // this hace referencia a xhttp
    if (this.readyState == 4 && this.status == 200) {
        dataAsString = this.responseText;
        // dataAsXML = this.responseXML ;
    }
};
xhttp.open("GET", "http://server.com/api/example", true);
xhttp.send();
```

ECMAScript 6 (2015)

- Reemplazo de `XMLHttpRequest` por `fetch`.
- Soportado en la mayor parte de los navegadores desde 2017. No soportado en:
  - Internet Explorer.
  - Opera Mini.
  - Blackberry browser.

```
Promise<Response> fetch(resource [, init ]);
```

- Parámetros de entrada:
  - `resource`: recurso que se solicita (url, objeto Request).
  - `init`: configuración de la petición:
    - `method`: tipo de petición (GET, POST, PUT, DELETE...).
    - `headers`: cabeceras que se incluyen en la petición.
    - `body`: cuerpo del mensaje (Blob, FormData, BufferSource, ReadableStream...).
    - `mode`: modo en el que se hará la petición (cors, no-cors, same-origin).
    - `credentials`: credenciales para realizar la petición.
    - `cache`: modo de cache.
    - `redirect`: modo de redirección usado.
- Valores de retorno:
  - `Promise` que se resuelve en un objeto `Response`:
    - Es un proxy que permite asociar código para gestionar qué hacer si la función asíncrona termina de forma correcta o devuelve un código de error.
    - Dispone de tres métodos:
      - `promise.then()`: se ejecuta cuando la petición asíncrona termina de forma exitosa.
      - `promise.catch()`: se ejecuta cuando se produce una excepción.
      - `promise.finally()`: se ejecuta al final, independientemente del resultado de la petición asíncrona.
  - Propiedades de `Response`:
    - `ok`: booleano que indica si la respuesta fue exitosa.
    - `status`: código de estado de la respuesta (200, 400, 404...).
    - `statusText`: mensaje correspondiente al código de estado.
    - `headers`: cabeceras.
    - `type`: tipo de la respuesta.

        - `url`: url de la respuesta.
    - Métodos de `Response`: Leen el *stream* de la respuesta y devuelven una `Promise` que se resuelve en otro objeto.
        - `arrayBuffer()`: devuelve una `Promise` a un `ArrayBuffer`.
        - `blob()`: devuelve una `Promise` a un `Blob`.
        - `formData()`: devuelve una `Promise` a un `FormData`.
        - `json()`: devuelve una `Promise` a un `JSON`.
        - `text()`: devuelve una `Promise` a un `USVString` (texto).

WAI-ARIA (*Web Accessibility Initiative - Accessible Rich Internet Applications*)

Con la aparición de la Web 2.0 las páginas web empezaron a utilizar JavaScript en masa y aparecieron cada vez más componentes no estándar, peticiones asíncronas, apareció AJAX... Todo esto complicaba el trabajo de las tecnologías de asistencia impidiendo interpretar las páginas de forma óptima.

Elementos

Se utilizan los elementos WAI-ARIA para añadir semántica a elementos html utilizados de manera diferente a la intención con la que fueron creados.

Roles

Identifican zonas de la página ayudando a las tecnologías asistivas a interpretarlas mejor.

Tipos:

- *Abstract roles*.
- *Landmark roles*.
- *Document structure roles*.
- *Widget roles*.
- *Live region roles*.

Landmark roles

Establecen marcas de navegación para lectores de pantalla e identifican regiones en el documento:

- `banner`.
- `complementary`.
- `contentinfo`.
- `form`.
- `main`.
- `navigation`.
- `region`.
- `search`.

Equivalencias semánticas.

Rol WAI-ARIA	Etiqueta HTML5
banner	header
complementary	aside
contentinfo	footer
main	main
navigation	nav

Se recomienda utilizar siempre las etiquetas HTML5.

Document structure roles

Describen estructuras que organizan el contenido de una página y no suelen ser interactivos.:

- `application`.
- `article`.
- `cell`.
- `columnheader`.
- `definition`.
- `directory`.
- `document`.
- `feed`.
- `figure`.
- `group`.
- `heading`.
- `img`.
- `list`.
- `listitem`.
- `math`.
- `none`.
- `presentation`.
- `region`.
- `row`.
- `rowgroup`.
- `rowheader`.
- `separator`.
- `table`.
- `term`.
- `toolbar`.

Existen dos formas de crear el mismo contenido visual:

- Etiquetas HTML nativas:

```
<ul>
    <li>Elemento lista 1</li>
    <li>Elemento lista 2</li>
    <li>Elemento lista 3</li>
</ul>
```

- Etiquetas HTML + CSS:

```
<div class="list">
    <p class="item">Elemento lista 1</p>
    <p class="item">Elemento lista 2</p>
    <p class="item">Elemento lista 3</p>
</div>
```

Sólo se incluyen roles para añadir semántica a las etiquetas que la necesitan:

```
<div class="list" role="list">
    <p class="item" role="listitem">Elemento lista 1</p>
    <p class="item" role="listitem">Elemento lista 2</p>
    <p class="item" role="listitem">Elemento lista 3</p>
</div>
```

Widget roles

Un *widget* es un objeto de la interface con el cual el usuario puede interactuar. Los roles de *widget* se usan para definir objetos no estándar en HTML:

- Widgets simples:
    - `alert`.
    - `alertdialog`.
    - `button`.
    - `checkbox`.
    - `dialog`.
    - `gridcell`.
    - `link`.
    - `log`.
    - `marquee`.
    - `menuitem`.
    - `menuitemcheckbox`.
    - `menuitemradio`.
    - `option`.
    - `progressbar`.
    - `radio`.
    - `scrollbar`.
    - `searchbox`.
    - `spinbutton`.
    - `status`.
    - `switch`.
    - `tab`.

    - `tabpanel`.
    - `textbox`.
    - `timer`.
    - `tooltip`.
    - `treeitem`.
- Widgets compuestos:
    - `combobox`.
    - `grid`.
    - `listbox`.
    - `menu`.
    - `menubar`.
    - `tablist`.
    - `tree`.
    - `treegrid`.

Live region roles

Indican que el contenido de la región puede cambiar de forma dinámica sin que el usuario interaccione con ellas.

Roles de widgets que por defecto son regiones activas:

- `alert`.
- `log`.
- `marquee`.
- `status`.
- `timer`.

Estados y propiedades

Atributos de *widgets*

Estados: definen las condiciones actuales de un elemento. Deben ser actualizados con Javascript cuando cambien las condiciones.

- `aria-checked`.
- `aria-disabled`.
- `aria-expanded`.
- `aria-hidden`.
- `aria-invalid`.
- `aria-pressed`.
- `aria-selected`.

Propiedades: proporcionan un significado extra a un elemento.

- `aria-autocomplete`.
- `aria-errormessage`.
- `aria-haspopup`.
- `aria-label`.
- `aria-level`.
- `aria-modal`.

- `aria-multiline`.
- `aria-multiselectable`.
- `aria-orientation`.
- `aria-placeholder`.
- `aria-readonly`.
- `aria-required`.
- `aria-sort`.
- `aria-valuemax`.
- `aria-valuemin`.
- `aria-valuenow`.
- `aria-valuetext`.

Atributos de relación

Describen asociaciones entre elementos que no pueden ser directamente deducidas:

- `aria-activedescendant`.
- `aria-colcount`.
- `aria-colindex`.
- `aria-colspan`.
- `aria-controls`.
- `aria-describedby`.
- `aria-details`.
- `aria-errormessage`.
- `aria-flowto`.
- `aria-labelledby`.
- `aria-owns`.
- `aria-posinset`.
- `aria-rowcount`.
- `aria-rowindex`.
- `aria-rowspan`.
- `aria-setsize`.

Atributos de regiones activas

Estados:

- `aria-busy`.

Propiedades:

- `aria-atomic`.
- `aria-live [off | polite | assertive]`.
- `aria-relevant [additions | additions text | removals | text | all]`.

Reglas de uso

- Usar el elemento/atributo nativo de html.
- No cambiar la semántica de los elementos.
- Todos los controles nativos ARIA deben ser usables con el teclado.